Labo 4 Geavanceerde Computerarchitectuur

Arne Van der Stede en Syme Vandenbosch

Academiejaar 2025-2026
March 26, 2026

1 Introductie

In dit labo wordt er gekeken naar het voordeel van het gebruik van gecoalesceerd geheugen. Door het gebruik van geheugenbandbreedte kan een GPU veel sneller werken dan een CPU. Deze bandbreedte kan pas behaald worden wanneer threads gecoalesceerde geheugentoegang vertonen. In dit labo bekijken we twee aspecten:

- het belang van gecoalesceerde globale geheugentoegang (dit bij eenvoudige matrix operaties)
- het effect van verschillende geheugentypen (globale, shared en constant memory) op matrixvermenigvuldiging. Er wordt gekeken naar de tijdswinst.

Door de uitvoeringstijden te vergelijken kunnen we inzicht krijgen op wat de grootste tijdswinst kan opleveren. De tijden van de GPU worden telkens vergeleken met een CPU implementatie zodat het voordeel van de GPU afgeleid kan worden.

2 Code

```cpp
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
#include <iomanip>

// Copy Kernel
__global__ void copyKernel(const float* A, float* B, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < N && col < N) {
        int index = row * N + col;
        B[index] = A[index];
    }
}

// Transpose Kernel
__global__ void transposeKernel(const float* A, float* B, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < N && col < N) {
        B[col * N + row] = A[row * N + col];
    }
}

int main() {
    std::vector<int> sizes = {2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096};
    const int iterations = 100;

    std::cout << " Matrix | Copy BW     | Copy Time  | Transp BW   | Transp Time" <<
        std::endl;
    std::cout << "-------|-------------|------------|-------------|------------" <<
        std::endl;

    for (int N : sizes) {
        size_t size = N * N * sizeof(float);
        double totalBytes = 2.0 * N * N * sizeof(float);

        float *d_A, *d_B;
```

```
        cudaMalloc(&d_A, size);
        cudaMalloc(&d_B, size);

        cudaMemset(d_A, 1, size);

        // Timing
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        // Define grid and block dimensions
        dim3 blockSize(32, 32);
        dim3 gridSize((N + blockSize.x - 1) / blockSize.x, (N + blockSize.y - 1) /
            blockSize.y);

        // Warm-up
        copyKernel<<<gridSize, blockSize>>>(d_A, d_B, N);
        transposeKernel<<<gridSize, blockSize>>>(d_A, d_B, N);
        cudaDeviceSynchronize();

        // Copy Kernel
        cudaEventRecord(start);
        for (int i = 0; i < iterations; ++i) {
            copyKernel<<<gridSize, blockSize>>>(d_A, d_B, N);
        }
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float msCopy = 0;
        cudaEventElapsedTime(&msCopy, start, stop);
        float avgMsCopy = msCopy / iterations;
        double bwCopy = (totalBytes / (avgMsCopy * 1e-3)) / 1e9;

        // Transpose Kernel
        cudaEventRecord(start);
        for (int i = 0; i < iterations; ++i) {
            transposeKernel<<<gridSize, blockSize>>>(d_A, d_B, N);
        }
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float msTranspose = 0;
        cudaEventElapsedTime(&msTranspose, start, stop);
        float avgMsTranspose = msTranspose / iterations;
        double bwTranspose = (totalBytes / (avgMsTranspose * 1e-3)) / 1e9;

        std::cout << std::fixed << std::setprecision(2)
                  << std::setw(6) << N << " | "
                  << std::setw(8) << bwCopy << " GB/s | "
                  << std::setw(7) << avgMsCopy << " ms | "
                  << std::setw(8) << bwTranspose << " GB/s | "
                  << std::setw(7) << avgMsTranspose << " ms" << std::endl;

        // Free memory
        cudaFree(d_A);
        cudaFree(d_B);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }

    return 0;
}
```

Listing 1: Eerste code

```
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>

void copyCPU(const std::vector<float>& A, std::vector<float>& B, int N) {
    for (int row = 0; row < N; ++row) {
        for (int col = 0; col < N; ++col) {
            B[row * N + col] = A[row * N + col];
        }
    }
}

```

```cpp
void transposeCPU(const std::vector<float>& A, std::vector<float>& B, int N) {
    for (int row = 0; row < N; ++row) {
        for (int col = 0; col < N; ++col) {
            B[col * N + row] = A[row * N + col];
        }
    }
}

int main() {
    std::vector<int> sizes = {2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096};
    const int iterations = 100; // CPU is slower than GPU

    std::cout << " Matrix | Copy BW     | Copy Time  | Transp BW   | Transp Time" <<
        std::endl;
    std::cout << "-------|-------------|------------|-------------|------------" <<
        std::endl;

    for (int N : sizes) {
        size_t size = N * N * sizeof(float);
        double totalBytes = 2.0 * size;

        std::vector<float> h_A(N * N, 1.0f);
        std::vector<float> h_B(N * N, 0.0f);

        // Warm-up
        copyCPU(h_A, h_B, N);
        transposeCPU(h_A, h_B, N);

    //Copy
        auto startCopy = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            copyCPU(h_A, h_B, N);
        }
        auto endCopy = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> durationCopy = endCopy - startCopy;
        double avgMsCopy = durationCopy.count() / iterations;
        double bwCopy = (totalBytes / (avgMsCopy * 1e-3)) / 1e9;

    //Transpose
        auto startTrans = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            transposeCPU(h_A, h_B, N);
        }
        auto endTrans = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> durationTrans = endTrans - startTrans;
        double avgMsTrans = durationTrans.count() / iterations;
        double bwTrans = (totalBytes / (avgMsTrans * 1e-3)) / 1e9;

        std::cout << std::fixed << std::setprecision(2)
                  << std::setw(6) << N << " | "
                  << std::setw(8) << bwCopy << " GB/s | "
                  << std::setw(7) << avgMsCopy << " ms | "
                  << std::setw(8) << bwTrans << " GB/s | "
                  << std::setw(7) << avgMsTrans << " ms" << std::endl;
    }

    return 0;
}
```

```cpp
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
#include <iomanip>

#define TILE_WIDTH 16
#define MAX_N_CONST 128
__constant__ float const_A[MAX_N_CONST * MAX_N_CONST];

__global__ void GlobalToSharedKernel(const float* A, int N) {
    __shared__ float ds_A[TILE_WIDTH][TILE_WIDTH];
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int row = blockIdx.y * TILE_WIDTH + ty;
    int col = blockIdx.x * TILE_WIDTH + tx;

    if (row < N && col < N) {
        ds_A[ty][tx] = A[row * N + col];
    }
```

```
    __syncthreads();
}

// 1. Global Memory Kernel
__global__ void GlobalMemMatrixMul(const float* A, const float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) sum += A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}

// 2. Shared Memory Kernel
__global__ void SharedMemMatrixMul(const float* A, const float* B, float* C, int N) {
    __shared__ float ds_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_B[TILE_WIDTH][TILE_WIDTH];
    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y * TILE_WIDTH + ty, col = blockIdx.x * TILE_WIDTH + tx;
    float sum = 0.0f;
    for (int t = 0; t < (N + TILE_WIDTH - 1) / TILE_WIDTH; t++) {
        if (row < N && t * TILE_WIDTH + tx < N) ds_A[ty][tx] = A[row * N + t *
            TILE_WIDTH + tx];
        else ds_A[ty][tx] = 0.0f;
        if (col < N && t * TILE_WIDTH + ty < N) ds_B[ty][tx] = B[(t * TILE_WIDTH + ty) *
            N + col];
        else ds_B[ty][tx] = 0.0f;
        __syncthreads();
        for (int k = 0; k < TILE_WIDTH; k++) sum += ds_A[ty][k] * ds_B[k][tx];
        __syncthreads();
    }
    if (row < N && col < N) C[row * N + col] = sum;
}

// 3. Constant Memory Kernel
__global__ void ConstantMemMatrixMul(const float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) sum += const_A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}

int main() {
    std::vector<int> sizes = {4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096};
    const int iterations = 10;

    std::cout << std::fixed << std::setprecision(5);
    std::cout << " Matrix | CPU->Dev | Dev->Shared | Global Kern | Shared Kern |
        Constant Kern" << std::endl;
    std::cout <<
        "-------|----------|-------------|-------------|-------------|--------------"
        << std::endl;

    for (int N : sizes) {
        size_t size = N * N * sizeof(float);
        std::vector<float> h_A(N * N, 1.0f);
        float *d_A, *d_B, *d_C;
        cudaMalloc(&d_A, size); cudaMalloc(&d_B, size); cudaMalloc(&d_C, size);
        cudaMemcpy(d_A, h_A.data(), size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_B, h_A.data(), size, cudaMemcpyHostToDevice);

        dim3 blockSize(TILE_WIDTH, TILE_WIDTH);
        dim3 gridSize((N + blockSize.x - 1) / blockSize.x, (N + blockSize.y - 1) /
            blockSize.y);

        cudaEvent_t start, stop;
        cudaEventCreate(&start); cudaEventCreate(&stop);

        // 1. Migration: CPU -> GPU Global
        cudaEventRecord(start);
        for (int i = 0; i < iterations; i++) cudaMemcpy(d_A, h_A.data(), size,
            cudaMemcpyHostToDevice);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
```

```
        float msCPUtoDev;
        cudaEventElapsedTime(&msCPUtoDev, start, stop);

        // 2. Migration: GPU Global -> GPU Shared
        cudaEventRecord(start);
        for (int i = 0; i < iterations; i++) GlobalToSharedKernel<<<gridSize,
            blockSize>>>(d_A, N);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float msDevToShared;
        cudaEventElapsedTime(&msDevToShared, start, stop);

        // 3. Global Kernel
        cudaEventRecord(start);
        for (int i = 0; i < iterations; i++) GlobalMemMatrixMul<<<gridSize,
            blockSize>>>(d_A, d_B, d_C, N);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float msGlobal;
        cudaEventElapsedTime(&msGlobal, start, stop);

        // 4. Shared Kernel
        cudaEventRecord(start);
        for (int i = 0; i < iterations; i++) SharedMemMatrixMul<<<gridSize,
            blockSize>>>(d_A, d_B, d_C, N);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float msShared;
        cudaEventElapsedTime(&msShared, start, stop);

        // 5. Constant Kernel
        float msConst = -1.0f;
        if (N <= MAX_N_CONST) {
            cudaMemcpyToSymbol(const_A, h_A.data(), size);
            cudaEventRecord(start);
            for (int i = 0; i < iterations; i++) ConstantMemMatrixMul<<<gridSize,
                blockSize>>>(d_B, d_C, N);
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);
            cudaEventElapsedTime(&msConst, start, stop);
        }

        std::cout << std::setw(6) << N << " | "
                  << std::setw(8) << msCPUtoDev / iterations << " ms | "
                  << std::setw(10) << msDevToShared / iterations << " ms | "
                  << std::setw(10) << msGlobal / iterations << " ms | "
                  << std::setw(10) << msShared / iterations << " ms | ";
        if (msConst >= 0) std::cout << std::setw(11) << msConst / iterations << " ms" <<
            std::endl;
        else std::cout << "        N/A" << std::endl;

        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }
    return 0;
}
```

```
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>

void cpuMatrixMul(const std::vector<float>& A, const std::vector<float>& B,
    std::vector<float>& C, int N) {
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < N; ++k) {
                sum += A[i * N + k] * B[k * N + j];
            }
            C[i * N + j] = sum;
        }
    }
}

```

```cpp
int main() {
    std::vector<int> sizes = {4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096};
    const int iterations = 10;

    std::cout << " Matrix | Time (ms) " << std::endl;
    std::cout << "-------|------------" << std::endl;

    for (int N : sizes) {
        std::vector<float> h_A(N * N, 1.0f);
        std::vector<float> h_B(N * N, 1.0f);
        std::vector<float> h_C(N * N, 0.0f);

        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; i++) {
            cpuMatrixMul(h_A, h_B, h_C, N);
        }
        auto end = std::chrono::high_resolution_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        double avgMs = duration.count() / iterations;

        std::cout << std::setw(6) << N << " | "
                  << std::fixed << std::setprecision(4)
                  << std::setw(10) << avgMs << " ms" << std::endl;
    }

    return 0;
}
```

3 Resultaten

3.1 Terminal uitvoer

part1

GPU

Matrix	Copy BW	Copy Time	Transp BW	Transp Time
2	0.00 GB/s	0.01 ms	0.00 GB/s	0.01 ms
4	0.01 GB/s	0.02 ms	0.01 GB/s	0.02 ms
8	0.03 GB/s	0.02 ms	0.02 GB/s	0.02 ms
16	0.12 GB/s	0.02 ms	0.12 GB/s	0.02 ms
32	0.44 GB/s	0.02 ms	0.51 GB/s	0.02 ms
64	1.62 GB/s	0.02 ms	1.93 GB/s	0.02 ms
128	7.82 GB/s	0.02 ms	6.45 GB/s	0.02 ms
256	29.32 GB/s	0.02 ms	19.33 GB/s	0.03 ms
512	65.18 GB/s	0.03 ms	25.94 GB/s	0.08 ms
1024	90.76 GB/s	0.09 ms	27.46 GB/s	0.31 ms
2048	92.85 GB/s	0.36 ms	36.97 GB/s	0.91 ms
4096	96.21 GB/s	1.40 ms	47.45 GB/s	2.83 ms

Figure 1: Tijden voor GPU verwerkingen

CPU

Matrix	Copy BW	Copy Time	Transp BW	Transp Time
2	3.56 GB/s	0.00 ms	4.57 GB/s	0.00 ms
4	9.14 GB/s	0.00 ms	9.14 GB/s	0.00 ms
8	25.60 GB/s	0.00 ms	10.67 GB/s	0.00 ms
16	48.76 GB/s	0.00 ms	11.07 GB/s	0.00 ms
32	61.59 GB/s	0.00 ms	11.67 GB/s	0.00 ms
64	94.71 GB/s	0.00 ms	12.13 GB/s	0.00 ms
128	48.73 GB/s	0.00 ms	2.74 GB/s	0.05 ms
256	30.58 GB/s	0.02 ms	1.85 GB/s	0.28 ms
512	29.95 GB/s	0.07 ms	1.66 GB/s	1.26 ms
1024	21.65 GB/s	0.39 ms	1.99 GB/s	4.21 ms
2048	12.91 GB/s	2.60 ms	0.78 GB/s	43.23 ms
4096	14.16 GB/s	9.48 ms	0.66 GB/s	202.33 ms

Figure 2: Tijden voor CPU verwerkingen

part2

GPU

Matrix	CPU->Dev	Dev->Shared	Global Kern	Shared Kern	Constant Kern
4	0.14541 ms	0.18452 ms	0.09854 ms	0.02150 ms	0.01918 ms
8	0.05099 ms	0.01720 ms	0.03276 ms	0.01233 ms	0.01894 ms
16	0.07810 ms	0.02525 ms	0.03176 ms	0.02636 ms	0.02036 ms
32	0.14380 ms	0.05468 ms	0.03460 ms	0.04921 ms	0.05710 ms
64	0.16732 ms	0.08896 ms	0.07376 ms	0.02578 ms	0.02376 ms
128	0.15751 ms	0.06046 ms	0.08079 ms	0.06997 ms	0.11035 ms
256	0.27202 ms	0.08892 ms	0.21074 ms	0.13769 ms	N/A
512	0.37964 ms	0.09215 ms	1.70851 ms	1.04040 ms	N/A
1024	0.97632 ms	0.02382 ms	13.49670 ms	7.10653 ms	N/A
2048	3.76827 ms	0.04670 ms	69.52632 ms	43.15608 ms	N/A
4096	12.03535 ms	0.14891 ms	563.72217 ms	347.82486 ms	N/A

Figure 3: Tijden voor GPU verwerkingen bij vermenigvuldiging

CPU

Matrix	Time (ms)
4	0.0001 ms
8	0.0005 ms
16	0.0042 ms
32	0.0339 ms
64	0.2686 ms
128	2.4912 ms
256	22.4932 ms
512	214.1579 ms
1024	1984.7228 ms

Figure 4: Tijden voor CPU verwerkingen bij vermenigvuldiging

3.2 Grafiek

Copy

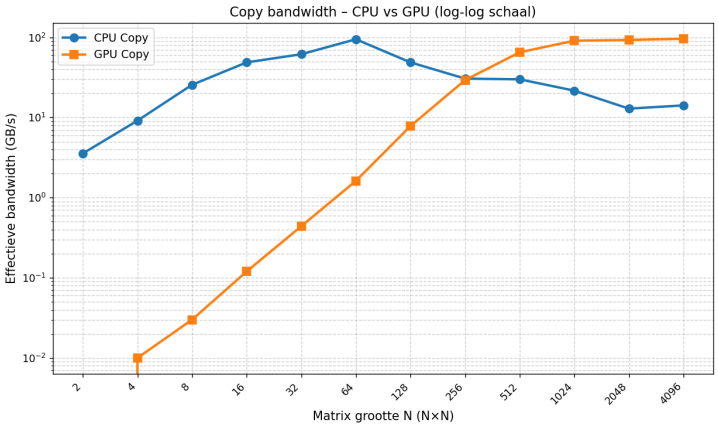


Figure 5: Vergelijking cpu vs gpu voor een copy operatie

Transpose

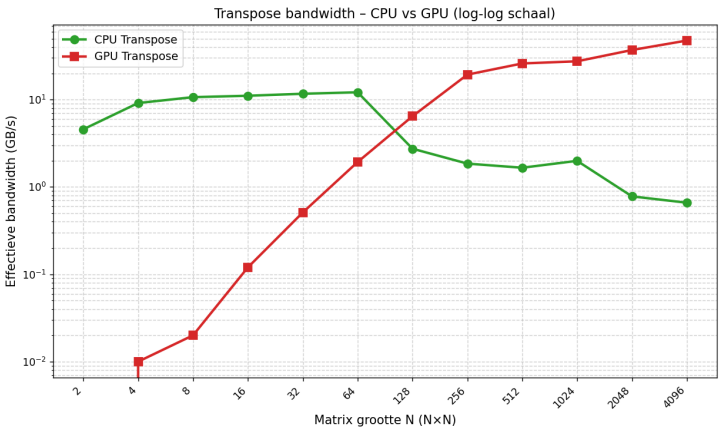


Figure 6: Vergelijking cpu vs gpu voor een transpose operatie

4 Guiding questions

4.1 How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?

- **A: CPU → GPU global memory**

  De tijd schaalt lineair met de matrixgroottes. Bij kleine matrices (< 256) is de tijd langer door de API overhead en de driver latency. Bij grotere matrices (> 1024) is de grotere bandbreedte die gebruikt kan worden duidelijk. Samengevat: bij kleine matrices domineert de opstartkost, bij grote matrices domineert de beschikbare PCIe-doorvoer.

- **B: GPU global → GPU shared memory**

  Hier wordt de snelheid waarmee globale memory kan worden gelezen en vervolgens in shared memory binnen een block kan worden geschreven bekeken. De schaling van tijdsversnelling is constant tot 1024, daarna is er een lichte toename. Hieruit wordt vooral duidelijk dat globale memory reads op de GPU zeer snel zijn (door het gebruik van coalescing). Schrijven tussen shared memory binnen een blok vraagt bijna geen extra tijd. Waardoor er kan besloten worden dat het schrijven van GPU global naar GPU shared memory minder tijd inneemt dan een CPU naar GPU. Het gebruik van shared memory in kernels is kan zorgen voor een grote tijdswinst.

4.2 How do the execution times of matrix multiplication compare when employing the three different memory types?

We vergelijken drie kernels voor C = A × B (naive global, tiled shared, constant memory):

Belangrijke observaties: Voor kleine matrices ($N < 64$) is elke kernel ongeveer even snel omdat het computationele overheerst. Vanaf ongeveer bij $N = 128$ wordt het duidelijk dat de shared memory sneller is. Constant memory is enkel bruikbaar tot $N = 128$. Het is soms sneller bij waardes zoals bij N=64, maar vanaf $N = 128$ is de bandbreedte te klein het geheugen zit vol. De shared memory kernel wint overduidelijk bij de grotere matrices. Er kan besloten worden dat shared memory bij (middel)grote matrices de beste keuze is.

4.3 How do the times scale with input and/or grid size?

Voor kleine $N < 128$ is de tijd bijna constant (Vooral door kernel launch overhead) de hoeveelheid werk is nog te klein om het geheugensysteem echt te belasten. Bij middelgrote N 128–512 is de overgangsfase. De tijdswinst bij de naive global kernel zakt snel weg. Voor grote $N > 1024$ schalen de verschillende kernels ongeveer als N^3, maar shared memory blijft opvallend sneller door de veel betere geheugen-efficiëntie.

5 Besluiten

Het belang van gecoalesceerd geheugentoegang en het juiste gebruik van de geheugentypen op GPU zijn onderzocht door eenvoudige matrixoperaties en matrixvermenigvuldiging.
Uit de resultaten wordt het duidelijk dat gecoalesceerde toegang absoluut noodzakelijk is om gebruik te kunnen maken van de geheugenbandbreedte. Bij de copy-operatie bereikt de GPU-bandbreedte bij de beste gevallen (kleine matrices) waarden tot 95 GB/s, terwijl de naive transpose-coalescing kernel inzakt tot 2 GB/s (bij grote matrices) door slechte coalescing en strided toegang.

Bij de matrixvermenigvuldiging worden de klassieke verwachte resultaten versterkt:

- Naive globale memory kernels presteert slecht bij grotere matrices. Doordat elke thread herhaaldelijk N elementen uit globale memory leest. Hierdoor wordt de kernel sterk geheugen gebonden. Bij grote N bereikt deze kernel slechts ongeveer 10-30% van de theoretische piekbandbreedte van het globale geheugen. Dit zorgt voor lange uitvoeringstijden.

- De constant memory kernel heeft bij zeer kleine matrices een prestatievoordeel. Dit komt door de efficiënte caching en broadcastingmechanismen van constant memory (alle threads tegelijk in één warp lezen). Constant memory is beperkt tot 64 kB, waardoor de aanpak enkel bruikbaar is tot $N = 128$. Vanaf 128 zakt de presatie al aanzienlijk (bandbreedtebeperking). En boven deze grens is het niet meer toepasbaar.

- De shared memory kernel lost het probleem dat optreed bij naive globale memory op. De data van A en B worden per tile eenmailg uit globale memory gehaald en vervolgens intensief hergebruikt binnen de shared memory van het blok. Hierdoor daalt het aantal globale memory accesses, terwijl computationeel de intensiteit wel toeneemt. Dit leidt tot een duidelijke versnelling van global memory naar shared memory.

6 Conclusie

Door het gebruik van de GPU bij matrixoperaties kan er een enorme tijdsprestatiewinst bekomen worden ten opzichte van de CPU. Vooral wanneer er coalescing memory en de juiste geheugen-hiërachie (shared memory) gebruikt wordt. Shared memory is de meest bepalende factor winstfactor bij het uitvoeren computationele kernels. Het is duidelijk dat een groot deel van de theoretische GPU prestaties onbenut blijven als niet het juiste geheugenbeheer toegepast wordt.